



KRN / TR-001

Kernellum Compiler v0.1: Constraint-Driven Generation of a Quantized Neural Accelerator

Sushanth Dasari - Kernellum Research
Version 1.0 · September 2026

STATUS	Technical report · v1.0	WORKLOAD	64 -> 32 -> 16 -> 10
SCOPE	Model -> search -> RTL	NEXT	Physical FPGA evidence

01 / ABSTRACT

Abstract

Kernellum Compiler v0.1 is a compact research prototype for model-to-accelerator generation. It trains a small neural workload, constructs an integer-only representation, evaluates candidate MAC-lane configurations against a latency constraint, emits FPGA-oriented SystemVerilog and memory initialization files, and verifies the generated design against a separate cycle-accurate software model. On a 64 -> 32 -> 16 -> 10 handwritten-digit classifier, the floating-point model reaches **96.22%** held-out accuracy and the generated INT8 execution model reaches **96.44%**; their predictions agree on **99.78%** of 450 held-out samples. The cycle-level model matches the vectorized INT8 output tensor exactly on **450/450** samples. The generated RTL passes **32/32** held-out golden-vector simulations under Icarus Verilog and synthesizes successfully under Yosys with **0 CHECK problems**. Under an assumed 100 MHz clock and a ≤ 10 us modeled core-latency constraint, the architecture search selects four parallel MAC lanes, corresponding to **680 modeled compute cycles / 6.8 us**. Physical FPGA implementation remains future work.

The 6.8 us figure is a cycle-count model at an assumed clock. It is not measured FPGA timing.

02 / EVIDENCE

Evidence levels

EVIDENCE	CURRENT STATUS
Float / INT8 functional evaluation	PASS
Independent cycle model	450 / 450 exact output tensors
Generated RTL simulation	32 / 32 golden cases PASS
Generic Yosys synthesis	PASS; 0 CHECK problems
Named FPGA family synthesis	ECP5 family-mapped synthesis available



EVIDENCE	CURRENT STATUS
Place-and-route / timing closure	not yet claimed
Board power / measured latency	not yet claimed

03 / WORKLOAD

Workload

FIELD	VALUE
Dataset	sklearn.datasets.load_digits
Network	64 -> 32 -> 16 -> 10
Hidden activation	ReLU
Held-out samples	450
Float accuracy	96.22%
INT8 accuracy	96.44%
Float / INT8 prediction agreement	99.78%

04 / SEARCH

Architecture search

MAC LANES	CORE CYCLES	MODELED LATENCY @100 MHz	MEETS <=10 us
1	2720	27.2 us	no
2	1360	13.6 us	no
4	680	6.8 us	yes - selected
8	340	3.4 us	yes
16	170	1.7 us	yes

The selected 4-lane configuration is the smallest candidate in the current search set that satisfies the modeled latency target.

05 / SYNTHESIS

Generic synthesis evidence

The public CI flow completes generic Yosys synthesis and reports approximately **37,039 wires**, **38,317 wire bits**, **38,170 generic cells**, **0 processes after synthesis**, and **0 Yosys CHECK problems**. These are generic post-synthesis logic cells, not named-device LUT/FF/DSP/BRAM utilization.

The repository also exposes ECP5 family-mapped synthesis for the current design. Family mapping still does not establish place-and-route timing closure or measured board performance.

06 / RTL

Generated hardware and verification

The generated accelerator uses signed INT8 activations and weights, 32-bit accumulators, four MAC lanes, three dense layers, signed requantization, saturation/ReLU behavior, memory-initialized parameters, input writes, output logits, and start/busy/done control. The testbench evaluates 32 held-out cases against generated golden



vectors.

07 / RERUN

Reproduce

```
pip install -e '[test,onnx]' pytest -q python -m kernellum --out artifacts/digits_int8 bash
scripts/run_eda.sh bash scripts/run_ecp5.sh python scripts/build_onnx_demo.py bash
scripts/run_onnx_eda.sh
```

08 / BOUNDARY

Claim boundary

v0.1 does not establish physical FPGA timing closure, post-route named-board utilization, measured power, measured end-to-end latency, arbitrary ONNX support, transformer operators, ASIC readiness, or competitive superiority over established accelerator compilers.

09 / NEXT

Next milestone

v0.2 alpha begins ONNX graph lowering, an explicit hardware IR and a Lattice ECP5-85F target profile. The next evidence threshold is place-and-route on a named physical board followed by measured latency and power.

10 / CITATION

Citation

Dasari, S. (2026). *Kernellum Compiler v0.1: Constraint-Driven Generation of a Quantized Neural Accelerator*. Kernellum Research Technical Report 001, v1.0.

Repository: <https://github.com/sushxnthd/kernellum>